

IR Sensors

IEEE @ UCI

Announcements

- Check #the-lab in the IEEE at UCI Discord for when the lab is open!!!
 - Feel free to solder your mouse anytime the lab is open
 - An officer will be able to help you setup a soldering station if necessary!
- Aim to **finish** soldering your mouse by Week 7-8!!!
- Leave it in lab on the Micromouse shelf or take your kit home with you



What is an Infrared emitter (IR emitter)?

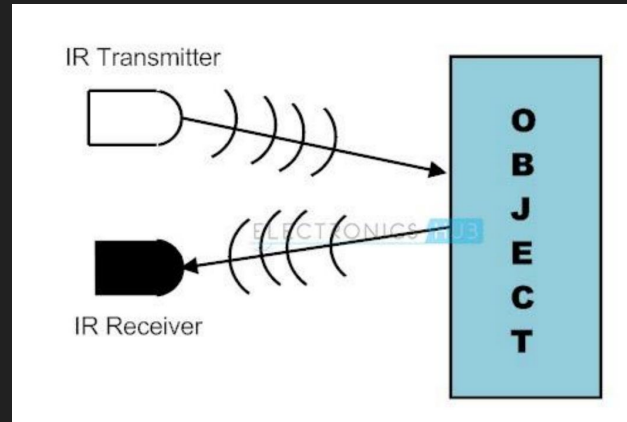


- A device that emits infrared (IR) light
- Often implemented using IR light-emitting diodes (LEDs)

What is an Infrared receiver (IR receiver)?



- Receive incoming infrared lights
- We are going to use IR emitters in conjunction with IR receivers to measure the distance between the mouse and walls



What IR emitter do we use?

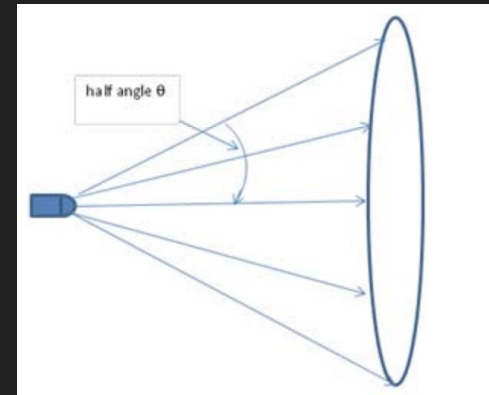


- SFH4545, [datasheet](#)
- Typical forward voltage: $V_F = 1.5 \text{ V}$
- Forward current: $I_{F \text{ MAX}} = 100 \text{ mA}$
- Half viewing angle: $\theta = 5^\circ$
- IR LED
 - Diode in forward biased condition

Diode Operating Functions:

- Forward-biased
 - allows current to flow
- Reverse-biased
 - Diode acts as insulator and doesn't allow current to flow

Relevant classes: EECS 170A



What IR receiver do we use?



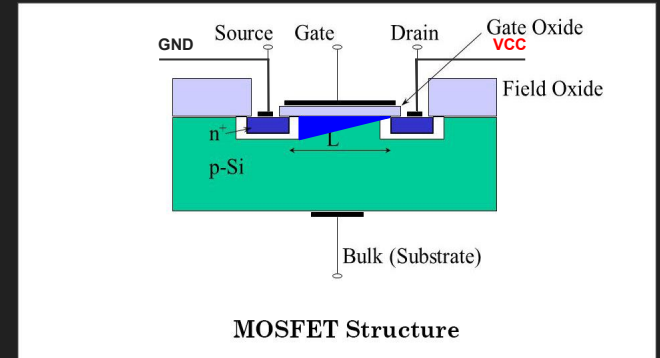
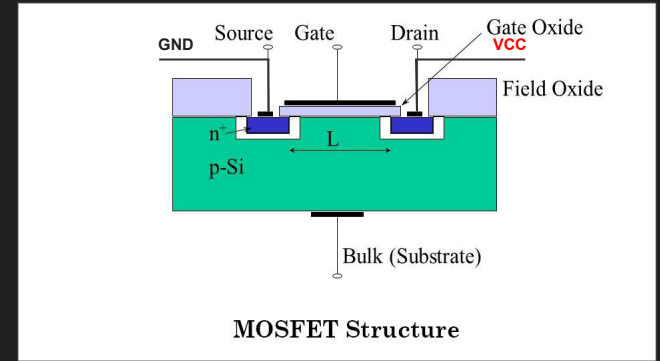
- TEFT4300, [datasheet](#)
- Collector current: $I_C = 50 \text{ mA}$
- Collector peak current: $I_{C \text{ MAX}} = 100 \text{ mA}$
- Half viewing angle: $\theta = 30^\circ$

Note:

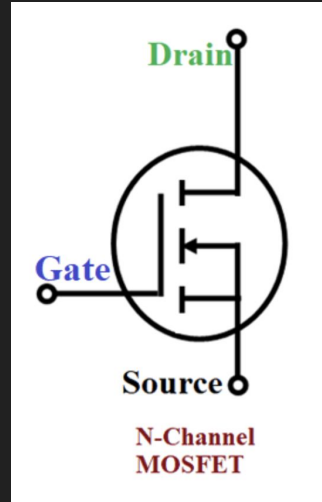
- Both emitters and receivers use the same polarity convention
 - Long end is negative (-)
 - Short end is positive (+)
 - Always check the datasheet

MOSFET (Metal Oxide Semiconductor Field Effect Transistor)

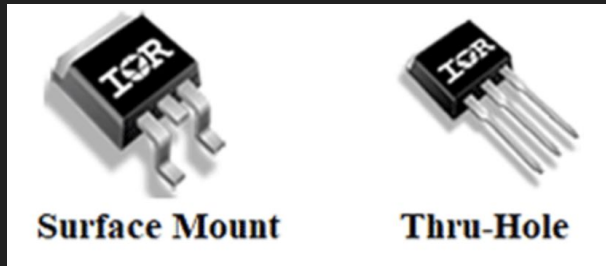
- Acting as a switch
- When voltage applied Gate, channel forms along L , allowing current to flow.
 - Must apply at least the minimum voltage stated on datasheet
- What happens to the channel when there is no voltage is applied to the Gate?



How do we turn the emitter on?

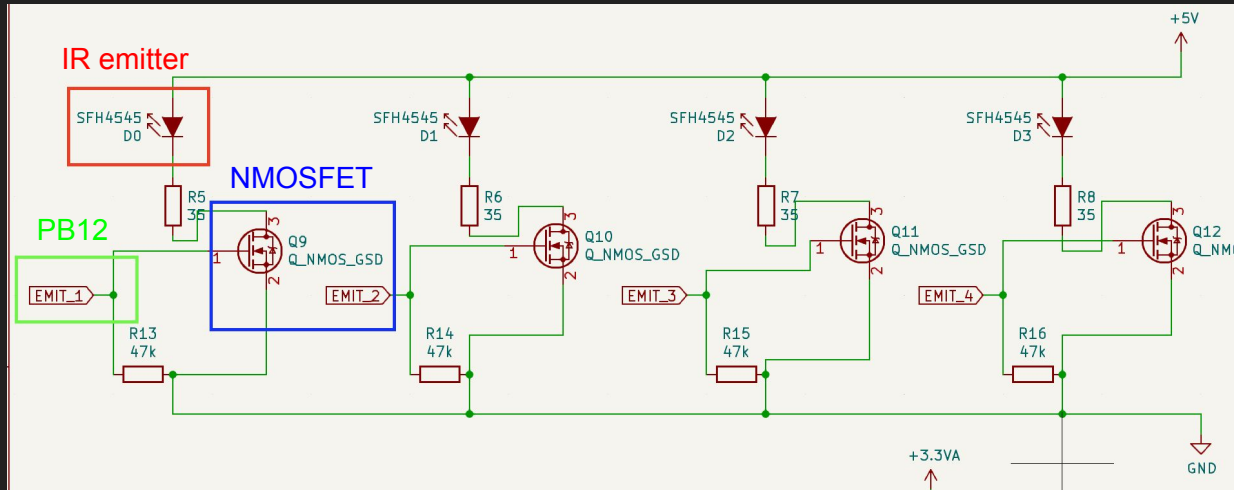


- We use a three-terminal N-Channel MOSFET
 - 1: Drain (D)
 - 2: Source (S) - connected to GND
 - 3: Gate (G)
- Current flows from drain to source when voltage is applied at the gate terminal
- Relevant classes: CSE 112, EECS 170A, EECS 170B

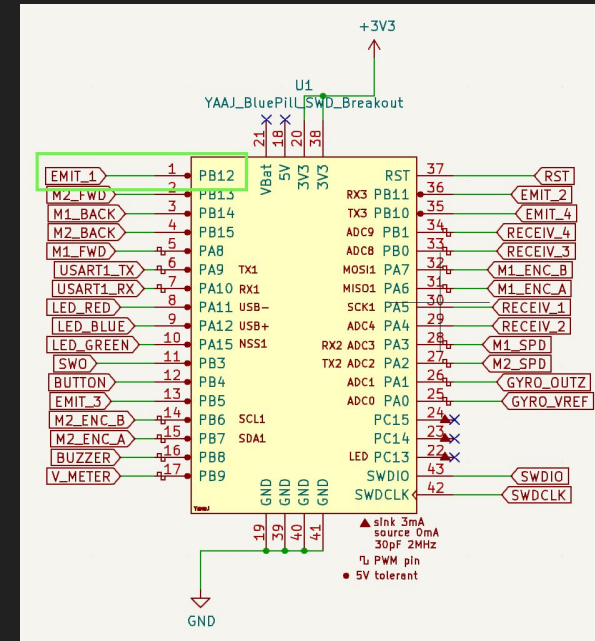


How do we turn the emitter on using the microcontroller?

- Schematic - IR emitters

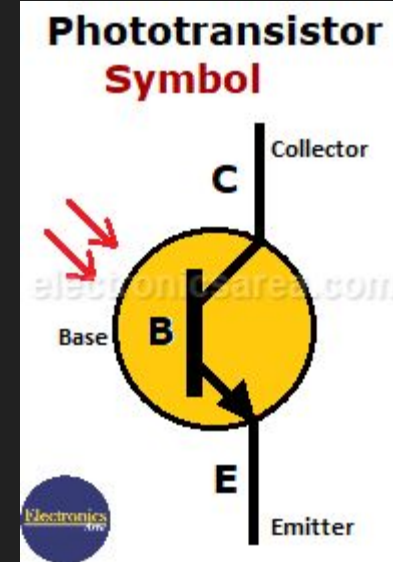
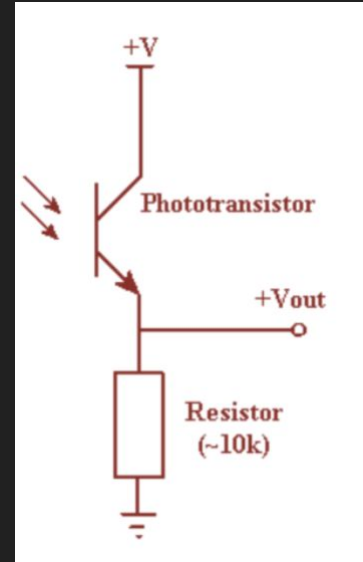


- Schematic - MCU

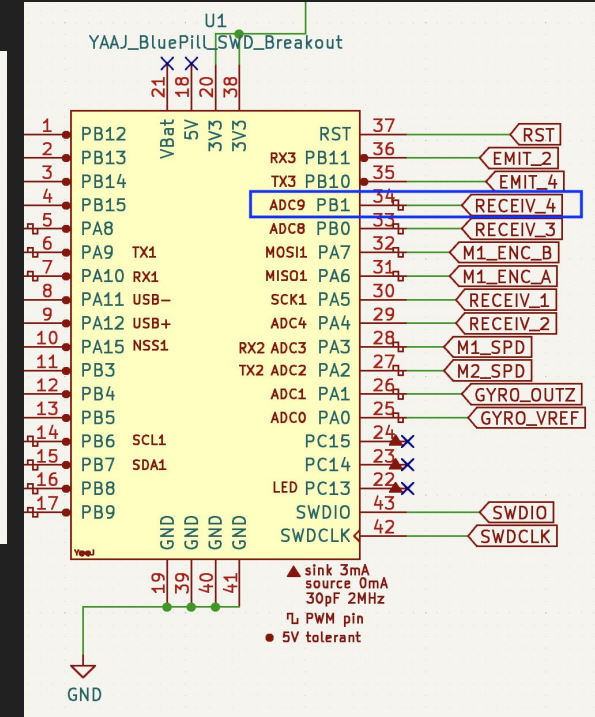


How do we receive the signal?

- NPN Phototransistor
 - Base - light-sensitive
 - Collector
 - Emitter
- Light energy (from IR emitters) is the input
- Current flows from the collector to the emitter region
- Current is proportional to the luminance
- Calculate distance based on the values read at V_{out}
- Outputs analog signals
 - Continuous (time-varying) signal, usually of voltage or current.



- Schematic - Receiver

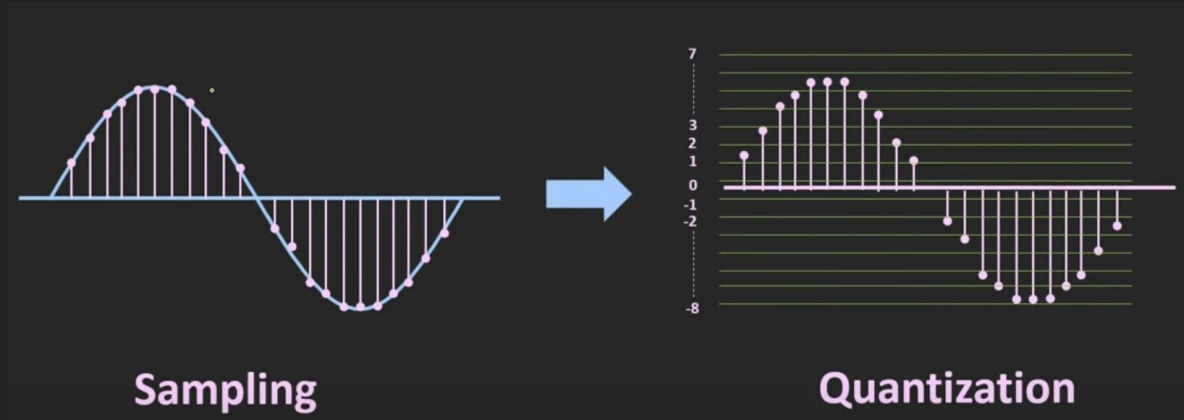


ADC Explanation



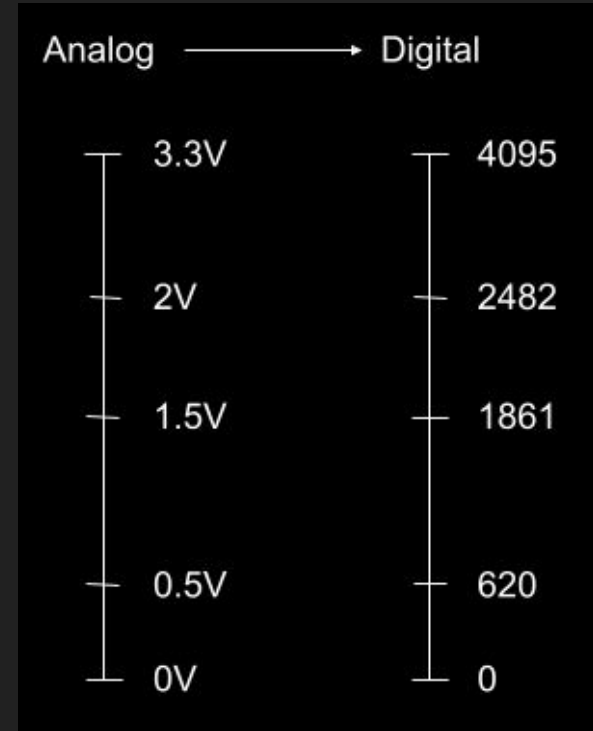
ADC - Analog to Digital Converter

1. The analog signal is sampled at a particular rate.
2. The sampled signal is quantized.
3. Encode the quantized signal into the binary format.

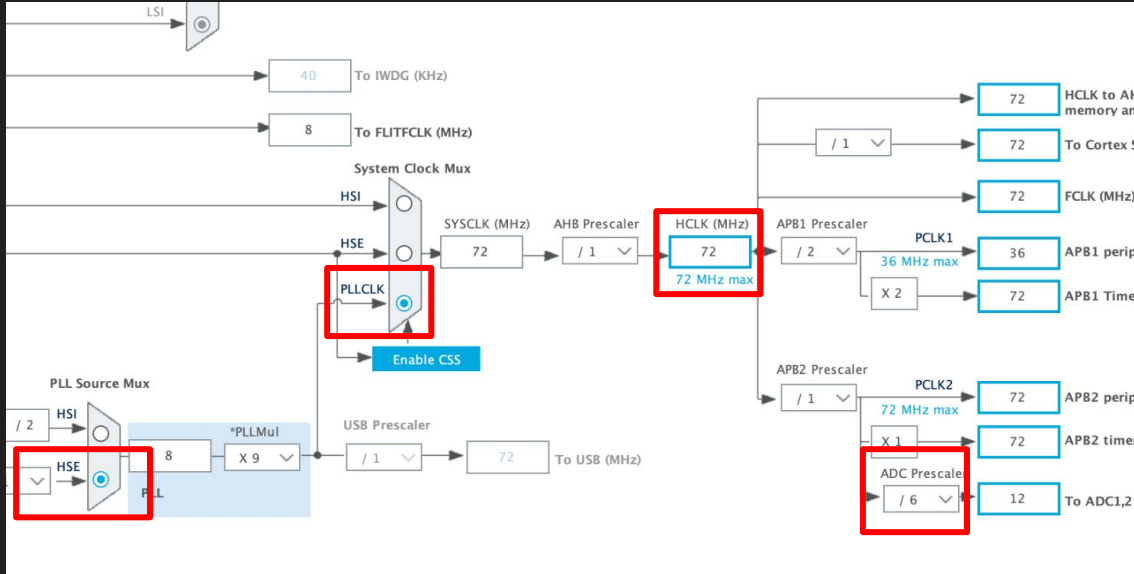


ADC in STM32F103C8T6

- 10 Analog Inputs on the STM32 Blue Pill Board
 - PA0 to PA7 (ADC0 to ADC7) and PB0 and PB1 (ADC8 and ADC9)
- ADC in the blue pill has a resolution of 12 bits.
 - The range of ADC values is from 0 to $2^{12} - 1$ (i.e. 0 to 4095).
- The smallest change in voltage that can be detected
 - $\text{Voltage / Step} = \text{Reference Voltage} / 2^{12}$
 - $3.3 / 4096 = 0.8056 \text{ mV / Step}$



Software Implementation - Clock Configuration



- Configure the clock to use the MCU's full potential - 72MHz

SYS Mode and Configuration	
Mode	
Debug	Serial Wire
☐ System Wake-Up	
Timebase Source	SysTick

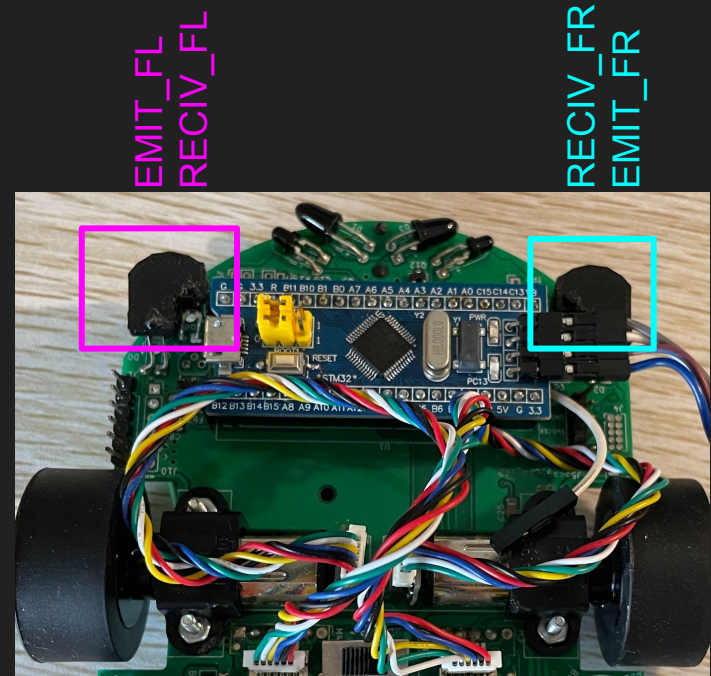
- Set SYS Mode

RCC Mode and Configuration	
Mode	
High Speed Clock (HSE)	Crystal/Ceramic Re...
Low Speed Clock (LSE)	Disable
☐ Master Clock Output	

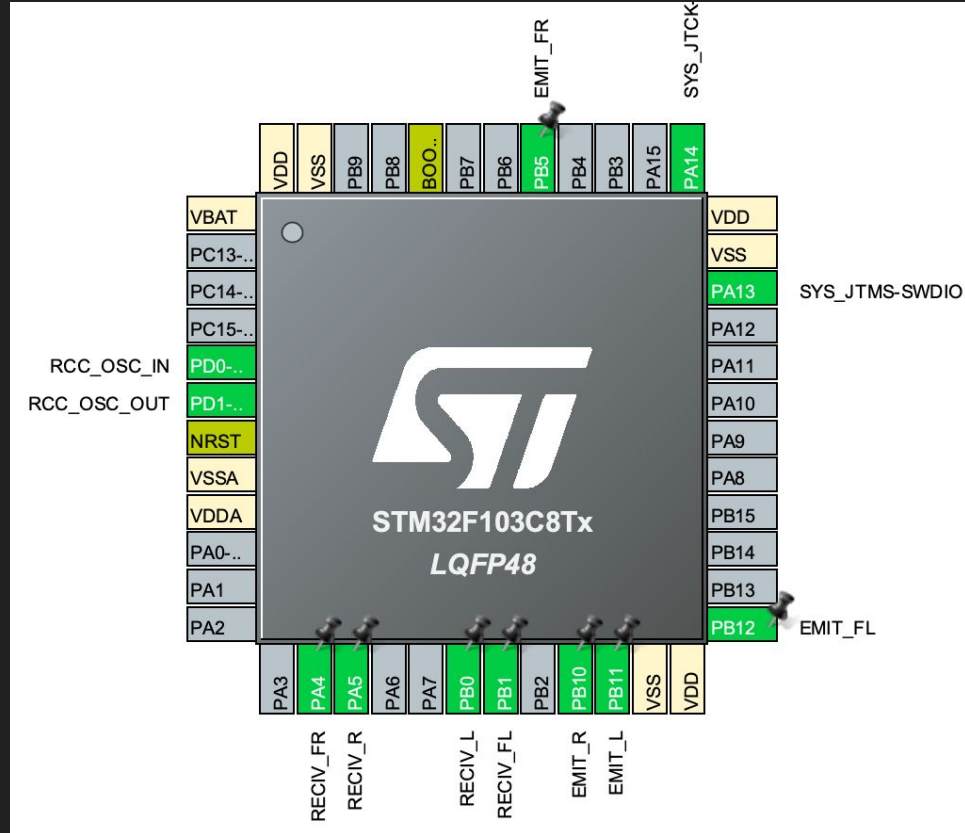
- Set RCC Mode

Software Implementation - Emitters & Receivers

- Define **PB12** as **GPIO_Output**, then **Right Click** to choose **Enter User Label**, and set the label as **EMIT_FL**
- Define **PB5** as **GPIO_Output**, then **Right Click** to choose **Enter User Label**, and set the label as **EMIT_FR**
- Define **PB1** as **ADC1_IN9**, then **Right Click** to choose **Enter User Label**, and set the label as **RECIV_FL**
- Define **PA4** as **ADC1_IN4**, then **Right Click** to choose **Enter User Label**, and set the label as **RECIV_FR**



Software Implementation - Emitters & Receivers



Software Implementation - Emitters & Receivers

- Save the .ioc file to generate code
- We can see pin definitions in “main.h”

```
#define RECIV_FR_Pin GPIO_PIN_4
#define RECIV_FR_GPIO_Port GPIOA
#define RECIV_FL_Pin GPIO_PIN_1
#define RECIV_FL_GPIO_Port GPIOB
#define EMIT_FL_Pin GPIO_PIN_12
#define EMIT_FL_GPIO_Port GPIOB
#define EMIT_FR_Pin GPIO_PIN_5
#define EMIT_FR_GPIO_Port GPIOB
```

- In **static void MX_ADC1_Init(void)**, code example for selecting ADC channels

```
sConfig.Channel = ADC_CHANNEL_4;
sConfig.Rank = ADC_REGULAR_RANK_1;
sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;
if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
{
    Error_Handler();
}
```

Software Implementation - Switching Channels

```
static void ADC1_Select_CH4(void) {
    ADC_ChannelConfTypeDef sConfig = {0};

    sConfig.Channel = ADC_CHANNEL_4;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_7CYCLES_5;
    if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
    {
        Error_Handler();
    }
}

static void ADC1_Select_CH9(void) {
    ADC_ChannelConfTypeDef sConfig = {0};

    sConfig.Channel = ADC_CHANNEL_9;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_7CYCLES_5;
    if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
    {
        Error_Handler();
    }
}
```

- We will call ADC1_Select_CH4 or ADC1_Select_CH9 to select the corresponding ADC channel when activating an IR sensor pair

Need to write

Software Implementation - typedef & variables

```
/* Private typedef -----  
/* USER CODE BEGIN PTD */  
typedef enum {  
    DIST_FL,  
    DIST_FR  
} dist_t;  
/* USER CODE END PTD */
```

Need to write

```
/* USER CODE BEGIN PV */  
uint16_t dis_FL;  
uint16_t dis_FR;  
/* USER CODE END PV */
```

Need to write

- Defined as public variables to monitor values in the debugging mode

```
/* Private variables -----  
ADC_HandleTypeDef hadc1;
```

Provided

- A data structure associated with the blue pill for configuring and controlling the ADC peripheral

Software Implementation - HAL functions

Useful functions we are going to use:

- `HAL_ADC_Start(&hadc1)` // start the ADC conversion
- `HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)` // with polling to check if the conversion is complete
- `HAL_ADC_GetValue(&hadc1)` // obtain the converted ADC value
- `HAL_ADC_Stop(&hadc1)` // stop the ADC conversion

Software Implementation - measure distance

Turn on the IR emitter



```
uint16_t measure_dist(dist_t dist) {
    GPIO_TypeDef* emitter_port;
    uint16_t emitter_pin;
    GPIO_TypeDef* receiver_port;
    uint16_t receiver_pin;

    switch(dist) {
        case DIST_FL:
            emitter_port    = EMIT_FL_GPIO_Port;
            emitter_pin     = EMIT_FL_Pin;
            receiver_port   = RECIV_FL_GPIO_Port;
            receiver_pin    = RECIV_FL_Pin;
            ADC1_Select_CH9();
            break;
        case DIST_FR:
            emitter_port    = EMIT_FR_GPIO_Port;
            emitter_pin     = EMIT_FR_Pin;
            receiver_port   = RECIV_FR_GPIO_Port;
            receiver_pin    = RECIV_FR_Pin;
            ADC1_Select_CH4();
            break;
        default:
            break;
    }
}
```

```
HAL_GPIO_WritePin(emitter_port, emitter_pin, GPIO_PIN_SET);
HAL_Delay(5);

HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
uint16_t adc_val = HAL_ADC_GetValue(&hadc1);
HAL_ADC_Stop(&hadc1);

HAL_GPIO_WritePin(emitter_port, emitter_pin, GPIO_PIN_RESET);

return adc_val;
}
/* USER CODE END 0 */
```

Read the ADC value

Turn off the IR emitter



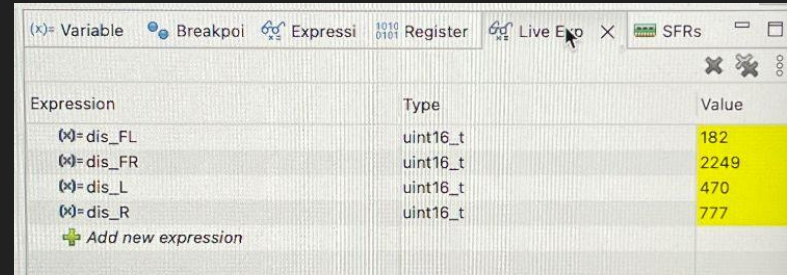
Select the IR pair and the channel



Software Implementation - while loop

- In while (1) loop, call measure_dist to get the IR values

```
while (1)
{
    dis_FR = measure_dist(DIST_FR);
    dis_FL = measure_dist(DIST_FL);
    HAL_Delay(1000);
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
```



Expression	Type	Value
(*)= dis_FL	uint16_t	182
(*)= dis_FR	uint16_t	2249
(*)= dis_L	uint16_t	470
(*)= dis_R	uint16_t	777
+ Add new expression		

- In Debug mode, show Live Expression view to see the values of dis_FR and dis_FL
- dis_FR and dis_FL are raw ADC values, need to convert them to distance values that are meaningful
- Could set IR_normals and IR_calibrations for this purpose
- Helpful to adjust errors when implementing PID controls